

QC Grade Service — How It Works

File: `qc_grade_service_3.js` **Main export:** `validateBobbinQC(bobbinNo)`

1. Purpose

Given a fiber-optic bobbin number, this service looks up the QC measurements taken for that bobbin, compares them against the specification "tiers" (grades) defined for its product type, and returns the best grade the bobbin qualifies for — or explains why it failed / what data is still missing.

It also contains special-case logic to automatically **upgrade or convert product types** when measurements fall within a secondary product's spec (referred to in the code as "D to A1 conversion").

2. Step-by-Step Flow

Step 1 — Fetch the measurement record

It queries `qc_entry_temp` for the row matching `bobbin_no`. If nothing is found, it returns an `ERROR` status immediately.

Step 2 — Determine fiber type from bobbin color

It looks up `fiber_color` from `bobbin_entries` and classifies the bobbin as:

`NATURAL` — color is exactly "NATURAL"

`RING_MARK` — color starts with "RM "

`COLORED` — anything else

This `fiber_type` is later used to pick the correct spec set (`color_type`) when the bobbin is **not** eligible for dual-product conversion.

Step 3 — Check the "D to A1 conversion" feature flag

It reads a flag (`enable_d_to_a1_conversion`) from an `app_config` table. This is an admin-controlled on/off switch for the whole conversion feature.

Step 4 — Decide if the bobbin qualifies for the secondary product spec

If the flag is on, `qualifiesForSecondaryProduct()` checks whether:

the bobbin's `product_type` is `G652D250`, **and**

its `mac_value` is below 7.05, **and**

a set of optical/mechanical measurements (cut-off wavelength, MFD, secondary coating diameter, fiber curl, cladding ovality) fall inside pre-defined ranges, checking the *top* and *bottom* reading of the fiber wherever both exist (`validateTopBottom`).

If all of that passes, `useDualProductTypeSpecs = true` and the "effective" product type becomes `G657A1250C` (`SECONDARY_PRODUCT_TYPE_FOR_LOW_MAC`) for the purposes of picking which mandatory parameters to check.

Step 5 — Load mandatory parameters and the "full check" flag

`mandatory_params` is pulled from `grade_mandatory` for the effective product type — this is the list of measurement fields that must be present/checked.

`full_check` is pulled from `pt_entry` for this bobbin — a per-bobbin flag saying whether both top and bottom readings are required.

Step 6 — Record which top/bottom pairs were fully measured (before any borrowing)

Before any values get borrowed (Step 7), the code snapshots — for every pair in `topBottomPairs` — whether **both** the top and bottom reading were genuinely present at that point:



```
js
const bothSidesPresentMap = {}; // paramName (top or bottom) -> true/false
for (const pair of topBottomPairs) {
  const bothPresent = !isEmpty(measurement[pair.top]) && !isEmpty(measurement[pair.bottom]);
  bothSidesPresentMap[pair.top] = bothPresent;
  bothSidesPresentMap[pair.bottom] = bothPresent;
}
```

This has to happen *before* the borrowing step, because after borrowing, both sides always look "present" (one is just a copy of the other). This map is used later in Step 10 to decide between a "Retest" and a "Test from Bottom" recommendation.

Step 7 — Top/Bottom value synchronization (only when full_check is false)

Many parameters are measured at two points on the fiber ("top" and "bottom"), listed in topBottomPairs. If full_check is off and **one side is missing but the other is present**, the code copies the present value into the missing one (in memory, and queues it for a DB write later). This lets a bobbin pass even if only one side was actually tested, using the tested side as a stand-in for the untested side.

Step 8 — Load the specification tiers

If the bobbin qualifies for dual-product specs, it fetches **both** the secondary product's tiers and the original product's tiers, secondary product first, each ordered by priority (1 = strictest/best grade). Otherwise it fetches only the original product's tiers, filtered additionally by fiber_type (color_type). If no active tiers exist, it returns an ERROR.

Step 9 — Check for missing data

After the top/bottom sync, it checks whether any of the mandatory_params are still null/empty. If so, it stops immediately and returns MISSING_DATA with the list of missing fields — no grading is attempted in this case, for any tier.

Step 10 — Grade matching loop

For each specification tier, from strictest to loosest:

Every mandatory parameter is checked against that tier's min_<param> / max_<param> bounds.

On the **first** failing parameter, the tier is abandoned; a failure log is recorded (which grade, which parameter, the measured value, the allowed range, and a recommendation). The recommendation is now:

"Retest" — the failing field is a top/bottom parameter **and** both its top and bottom readings were genuinely measured (from the Step 6 snapshot). Both sides were real data and it still failed, so the advice is to retest rather than trust a borrowed value.

"Test from Bottom" — the failing field is a top/bottom parameter but only one side was actually measured (the other was borrowed in Step 7).

"Standard parameter mismatch" — the field isn't a top/bottom parameter at all.

If every parameter passes, the tier is a candidate match — **but** if this tier belongs to the secondary product (dual-conversion path), there's an extra requirement:

All required MBend fields must be present (hasMbandValues) — if not, the function stops and returns MBEND_REQUIRED, telling the operator to complete MBend testing before a grade can be confirmed.

The MBend values must also pass their own min/max bounds (validateMband) — if they fail, this particular tier is skipped (continue) and the loop moves to the next tier (which may be a tier from the *original* product type).

Once a tier fully passes (including MBend, if applicable), it becomes the `finalMatchedTier` and the loop stops.

Step 11 — Persist results and finalize

If a tier matched:

Any top/bottom values that were borrowed in Step 7 are now written back to `qc_entry` (the permanent table, as opposed to the temp table used for grading).

Product type upgrade: if the bobbin's original type was already `G657A1250C`, or it qualified via the D-to-A1 conversion path, its `product_type` is updated to `G657A1250` in both `qc_entry_temp` and `bobbin_entries`.

Returns `PASSED` with the matched grade, its priority, and check counts.

If no tier matched at all, it returns `FAILED` with the details of the last failure encountered.

Error handling

Any unexpected exception is caught, logged, and returned as `CRITICAL_ERROR`. The database client is always released in a `finally` block.

3. Possible Return Statuses

Status	Meaning
<code>ERROR</code>	Bobbin or its bobbin_entries record not found, or no active spec tiers exist
<code>MISSING_DATA</code>	One or more mandatory parameters have no value (even after top/bottom borrowing)
<code>MBEND_REQUIRED</code>	Bobbin qualifies for the secondary product on optical specs, but MBend testing hasn't been entered yet
<code>PASSED</code>	A tier fully matched; grade, priority, and any product-type upgrade are applied
<code>FAILED</code>	No tier matched; details of the first failure on the last-checked tier are returned
<code>CRITICAL_ERROR</code>	An unexpected runtime/database error occurred

4. `failure_details.recommendation` — decision logic

Both top & bottom genuinely measured?	Parameter type	Recommendation
Yes	Top/bottom parameter	<code>"Retest"</code>
No (one side was borrowed)	Top/bottom parameter	<code>"Test from Bottom"</code>
n/a	Not a top/bottom parameter	<code>"Standard parameter mismatch"</code>

5. Key Concepts to Remember

Top/Bottom pairs — most physical measurements are taken at two points on the fiber; if `full_check` is off, a missing reading can be filled in from its pair. Whether both sides were *actually* measured is snapshotted before this borrowing happens, so the failure message can later tell a real double-sided failure (`"Retest"`) apart from a single-sided/borrowed one (`"Test from Bottom"`).

Tiers/priority — grades are checked strictest-first (`priority ASC`); the first tier that fully passes wins.

D-to-A1 (dual-product) conversion — a feature-flagged path that lets a `G652D250` bobbin with a low MAC value be evaluated against a *second*, better product's spec (`G657A1250C`), with an added MBend requirement, potentially upgrading it further to `G657A1250` on success.

Mandatory parameters are configurable per product type (`grade_mandatory` table) rather than hardcoded — except for the top/bottom sync list and MBend field list, which are hardcoded arrays in this file.